

CSE 4322

Software Project Management

EXAM 2 COMPREHENSIVE STUDY GUIDE

Chapters 7, 8, 13, 14 + Required Readings + Team Presentations

Format: 50 Questions | MCQ & True/False | 2 pts each | 100 pts total

Closed Book | Closed Notes | Respondus Lockdown Browser

Spring 2026 | Dr. Michael F. Siok | UTA

Contents

1	KEY CONCEPTS QUICK REFERENCE	2
1.1	Chapter 7: Software Cost Estimation	2
1.1.1	Three Algorithmic Models (Table 7.1)	2
1.1.2	COCOMO 81 (Barry Boehm, 63 TRW projects)	2
1.1.3	Walston-Felix	2
1.1.4	Putnam Model	2
1.1.5	Function Point Analysis (Albrecht, 1979)	2
1.1.6	COCOMO II (1995, Boehm) – Three Sub-Models	3
1.1.7	Key Principles	3
1.2	Chapter 8: Project Planning & Controls	4
1.2.1	Work Breakdown Structure (WBS)	4
1.2.2	PERT Chart (Program Evaluation and Review Technique)	4
1.2.3	Gantt Chart	4
1.2.4	Earned Value Management (EVM)	4
1.2.5	Risk Management	4
1.3	Chapter 13: Software Testing	6
1.3.1	Verification vs. Validation	6
1.3.2	Testing Levels (V-Model)	6
1.3.3	Black-Box vs. White-Box	6
1.3.4	Coverage Criteria (weakest → strongest)	6
1.3.5	Fagan Inspection	6
1.3.6	TDD (Test-Driven Development)	6
1.3.7	Mutation Testing	6
1.3.8	Regression Testing	6
1.4	Chapter 14: Software Maintenance	7
1.4.1	Maintenance Life Cycle	7
1.4.2	Effort Distribution	7
1.4.3	Lehman’s Laws	7
1.4.4	Key Definitions	7
1.4.5	Organization Types	7
1.4.6	Program Comprehension	7
1.4.7	Code Smells	8
1.5	Required Readings Key Points	9
2	PRACTICE EXAM QUESTIONS	10
2.1	Chapter 7: Software Cost Estimation	10
2.2	Chapter 8: Project Planning & Controls	13
2.3	Chapter 13: Software Testing	16
2.4	Chapter 14: Software Maintenance	19
3	ANSWER KEY – QUICK REFERENCE	21

KEY CONCEPTS QUICK REFERENCE

Chapter 7: Software Cost Estimation

Core Formula

Cost = Effort $\times$ Loaded Salary per Person-Month
 SW cost is dominated by **effort** (person-months), not equipment.
 Basic Model: Effort = $a + b(\text{kLOC})^c$, where kLOC = 1,000 lines of code.
 When $c > 1$: effort grows **more than linearly** with code size (most likely in practice).

Three Algorithmic Models (Table 7.1)

Model	Formula	Notes
Halstead	Effort = $0.7 \cdot \text{kLOC}^{1.50}$	
Boehm (COCOMO)	Effort = $2.4 \cdot \text{kLOC}^{1.05}$	
Walston-Felix	Effort = $5.2 \cdot \text{kLOC}^{0.91}$	Dubious: $c < 1$

COCOMO 81 (Barry Boehm, 63 TRW projects)

Class	b	c	Description
Organic	2.4	1.05	Small team, known environment, lots of experience
Semidetached	3.0	1.12	Between organic and embedded
Embedded	3.6	1.20	Inflexible environment, severe constraints

Three levels: **Basic** (crude), **Intermediate** (15 cost drivers), **Detailed** (cost drivers per phase).

Walston-Felix

29 variables influencing productivity, 3-level scale. Derived from 60 IBM projects. Model **unsatisfactory even on its own training data**.

Putnam Model

$MR(t) = 2Kae^{-at^2}$ (Rayleigh distribution). Rule of thumb: **40% development, 60% maintenance**. Best for very large projects (>15 man-years).

Function Point Analysis (Albrecht, 1979)

Does **NOT** require kLOC. Uses data structures to predict size.

5 components: Inputs (I), Outputs (O), Inquiries (E), Logical Internal Files (L), Interfaces (F)

UFP = $4I + 5O + 4E + 10L + 7F$ (average complexity weights)
 TCF = $0.65 + 0.01 \times \text{DI}$ (14 characteristics, rated 0–5, DI range: 0–70)
 FP = UFP $\times$ TCF

Best for **data-heavy business apps**, NOT for algorithm-heavy software (compilers, real-time).

COCOMO II (1995, Boehm) – Three Sub-Models

1. **Application Composition:** Uses Object Points (screens, reports, 3GL).
$$\text{NOP} = \text{ObjPts} \times (100 - \% \text{Reuse}) / 100. \quad \text{Effort} = \text{NOP} / \text{PROD}.$$
2. **Early Design:** Uses Function Points.
3. **Post-Architecture:** Similar to COCOMO 81 with updates.

Key Principles

- Any cost model must be **CALIBRATED** to your own environment.
- Cost models differ widely due to different definitions, tools, maturity, experience.
- **90/90 Rule** (Tom Cargill): First 90% of code = 90% of time; remaining 10% = other 90%.
- Software schedule slippage is **hard to detect** as it is happening.

Chapter 8: Project Planning & Controls

Work Breakdown Structure (WBS)

Hierarchical decomposition of project work. Lowest level = **work packages** with budget, dates, task manager.

Activity Constraints: Resource, Duration, Precedence.

PERT Chart (Program Evaluation and Review Technique)

DAG (Directed Acyclic Graph), activity-on-node. Developed by US Navy, 1950s.

- S_i = earliest start, F_i = earliest finish, L_i = latest finish.
- Slack = $L_i - F_i$. **Zero slack = CRITICAL PATH.**
- If critical path activity delays, **entire project delays.**

Gantt Chart

Henry Gantt, 1910s. Visual calendar representation of project schedule.

Earned Value Management (EVM)

EVM Definitions

PV (BCWS)	= Planned Value	= Budgeted Cost of Work <i>Scheduled</i>
EV (BCWP)	= Earned Value	= Budgeted Cost of Work <i>Performed</i>
AC (ACWP)	= Actual Cost	= Actual Cost of Work <i>Performed</i>
BAC	= Budget at Completion	= Total project budget

Variances & Indices

Variances:

$SV = EV - PV$ (negative = behind schedule)

$CV = EV - AC$ (negative = over budget)

Performance Indices:

$SPI = EV/PV$ (< 1 = behind schedule, > 1 = ahead)

$CPI = EV/AC$ (< 1 = over budget, > 1 = under budget)

Projections:

$EAC = BAC/CPI$ (estimated cost at completion)

$ETC = EAC - AC$ (estimate to complete)

$VAC = BAC - EAC$ (negative = over budget)

$TCPI = (BAC - EV)/(BAC - AC)$ (> 1 = over budget)

Risk Management

- Risk is neither bad nor good, but **always present**.
- Most projects do too **LITTLE** risk management.
- Risk Exposure = Probability $\times$ Effect.
- $CRE = (p_R \times E_R) + [(1 - p_R) \times E_D]$.

- Strategies: **Avoid, Transfer, Accept.**
- Top risk: **Personnel shortfall.** Risk officer = ideally NOT the PM.

Chapter 13: Software Testing

Error vs. Fault vs. Failure

Error = human mistake.

Fault = defect/bug in code (manifestation of error).

Failure = bad execution behavior.

Testing observes **FAILURES**, not faults directly.

1 failure may be caused by N faults; M failures may be caused by 1 fault.

Industry Stats: 30–85 errors per 1,000 LOC typical. Extensively tested: 0.5–3 per 1,000 LOC. 60% design errors, 40% implementation errors. 66% of design errors found only during operation. Cost of fixing errors increases **EXPONENTIALLY** with detection phase.

Verification vs. Validation

- **Verification** = Building the product **RIGHT** (against specs).
- **Validation** = Building the **RIGHT** product (customer needs).

Testing shows the presence of errors, **NOT** their absence. Exhaustive testing is infeasible.

Testing Levels (V-Model)

Unit → Integration → System → Acceptance → Regression

Black-Box vs. White-Box

- **Black-box** = functional/specification-based, no internal knowledge.
- **White-box** = structural/program-based, internal knowledge used.

Coverage Criteria (weakest → strongest)

Statement (All-Nodes) → Branch/Decision (All-Edges) → Condition/Decision → **MC/DC** → All-Paths

MC/DC = Modified Condition/Decision Coverage: each condition independently affects decision. Required by **DO-178** for safety-critical aviation software.

Fagan Inspection

~4 members: Moderator, Code Author (**silent**), 2 Inspectors/Readers.

Uses checklist. Very cost-effective for finding bugs.

TDD (Test-Driven Development)

Write tests **FIRST**, then code. Agile practice.

Mutation Testing

Insert known faults. Adequacy score = Dead mutants/Total mutants.

Competent Programmer Hypothesis + Coupling Effect Hypothesis.

Regression Testing

Retest after changes. Retest-all or selective retest.

Chapter 14: Software Maintenance

IEEE 610 Definition: Modifying software **AFTER delivery** to correct faults, improve performance, or adapt to changed environment.

Four Types of Maintenance

Type	%	Description
Corrective	21%	Fix bugs found after delivery
Adaptive	25%	Adapt to environment changes (new OS, HW). No functionality change
Perfective	50%	New/changed user requirements. LARGEST portion
Preventive	4%	Improve maintainability. Called Refactoring in Agile/XP

Maintenance Life Cycle

Introduction → Growth → Maturity → Decline.

Knowledge disappears during maintenance. Documentation becomes outdated. Original developers leave.

Effort Distribution

Isolation / determining what to change (**40%**) → Modification (30%) → Testing (30%)

Lehman's Laws

1. **Continuing Change:** System in use undergoes continuous change until replaced.
2. **Increasing Complexity:** Changes make programs less structured and more complex.

Key Definitions

- **Reverse Engineering** = Reconstruct design from existing code. Does **NOT** change system.
 - Design Recovery = higher abstraction level
 - Redocumentation = same abstraction level
- **Reengineering (Renovation)** = Reverse eng. + Forward eng. Functionality **DOES change**.
- **Restructuring (Refactoring)** = Same abstraction level, functionality does **NOT change**.

Maintenance requires **MORE skill than development** (must understand others' code + write new).

Organization Types

- **W-Type** (work type), **A-Type** (application domain), **L-Type** (life-cycle phase).
- **L-Type recommended** (separate dev and maintenance). Microsoft model.

Program Comprehension

Plans = program fragments implementing actions.

Beacons = key features (e.g., swap operation ⇒ sorting).

Code Smells

Long method, large class, duplicate code, feature envy, etc.

Required Readings Key Points

“Why Software Fails” (Charette): Common failure factors include unrealistic goals, inaccurate estimates, bad requirements, unmanaged risks, poor communication. 5–15% of projects abandoned. 40–50% of time on avoidable rework. Cost to fix errors compounds exponentially.

“We Did Nothing Wrong” (McCormick): The quality fight is never-ending and all software has bugs. In today’s software you have to assume there are always some bugs in the code. Case study of professional responsibility.

“Clueless and Oblivious” (Harrison): Kruger-Dunning Effect: incompetent people **overestimate** their abilities because they lack knowledge to recognize incompetence. CMM maturity leads to better predictability. Improvement may come from increased **AWARENESS**, not just process changes. PSP (Personal Software Process) by Watts Humphrey.

“Inside Legacy IT Systems”: No formal definition of legacy system. \$35 trillion spent on IT since 2010. COBOL still critical. Modernization is costly and risky. Legacy systems are organizational reality, not mistakes.

“Project Estimation with Use Case Points” (Clemmons):

$UCP = UUCP \times TCF \times ECF$, where $UUCP = UUCW + UAW$

$TCF = 0.6 + 0.01 \times TF$ (13 technical factors)

$ECF = 1.4 + (-0.03 \times EF)$ (8 environmental factors)

Effort = $UCP \times \text{Productivity Factor}$. Achieves within 20% accuracy.

Actor weights: Simple (API)=1, Average (protocol)=2, Complex (person/GUI)=3.

Use case weights: Simple=5, Average=10, Complex=15.

IEEE/ACM Code of Ethics: 8 principles: **Public**, Client/Employer, Product, Judgment, Management, Profession, Colleagues, Self. **Public interest is HIGHEST priority.** Requires honesty, integrity, competence. Obligation to report violations. Software only approved if safe, meets specs, passes tests.

PRACTICE EXAM QUESTIONS

50 Questions — MCQ & True/False — Matches Exam 1 format and difficulty

Chapter 7: Software Cost Estimation

1. In software development, the biggest portion of cost is effort (person-months), not equipment.

- a) True b) False

Answer: True – *No cranes, factories, or lab equipment needed. $Cost = effort \times loaded\ salary$.*

2. The Walston-Felix cost model has an exponent $c < 1$, which means effort grows less than linearly with code size. This is generally considered a reliable assumption.

- a) True b) False

Answer: False – $c < 1$ is dubious. Most models agree $c > 1$ (effort grows MORE than linearly).

3. In the basic cost model $Effort = a + b(kLOC)^c$, what does kLOC represent?

- a) 1,000 lines of code
b) 1,000 kilobytes of code
c) 1,000 function points
d) 1,000 test cases

Answer: a) 1,000 lines of code

4. Which COCOMO 81 project class has the HIGHEST exponent (c), indicating the most effort growth with project size?

- a) Organic
b) Semidetached
c) Embedded
d) All have the same exponent

Answer: c) Embedded – $c = 1.20$, **Semi:** $c = 1.12$, **Organic:** $c = 1.05$

5. Blindly applying an existing cost model without calibration will solve your cost estimation problem.

- a) True b) False

Answer: False – *Every cost model must be calibrated to your own SW engineering environment.*

6. Function Point Analysis was developed to address which limitation of algorithmic cost models?

- a) They require too many team members
b) They require kLOC as input, which is hard to estimate early
c) They cannot be used for web applications
d) They are too expensive to run

Answer: b) FP does NOT require kLOC; uses data structures instead.

7. In Function Point Analysis, the formula for Unadjusted Function Points (UFP) is:

- a) $UFP = 4I + 5O + 4E + 10L + 7F$

- b) $UFP = 3I + 4O + 3E + 7L + 5F$
- c) $UFP = I + O + E + L + F$
- d) $UFP = 5I + 7O + 5E + 15L + 10F$

Answer: a) Average complexity weights from Table 7.5

8. The Technical Complexity Factor (TCF) in Function Point Analysis is calculated as:

- a) $TCF = 0.65 + 0.01 \times DI$
- b) $TCF = 0.50 + 0.02 \times DI$
- c) $TCF = 1.00 + 0.01 \times DI$
- d) $TCF = DI/70$

Answer: a) 14 characteristics rated 0–5, DI range 0–70

9. Function Point Analysis is best suited for algorithm-heavy applications such as compilers and real-time software.

- a) True b) False

Answer: False – Best for data-structure-heavy business applications, NOT algorithm-heavy ones.

10. In COCOMO II, the Application Composition model uses which sizing metric?

- a) Lines of code
- b) Function Points
- c) Object Points (screens, reports, 3GL components)
- d) Use Case Points

Answer: c) Object = screen, report, or 3GL module

11. In COCOMO II Application Composition, the formula for New Object Points (NOP) is:

- a) $NOP = ObjPts \times (100 - \%Reuse)/100$
- b) $NOP = ObjPts \times \%Reuse/100$
- c) $NOP = ObjPts + \%Reuse$
- d) $NOP = ObjPts/PROD$

Answer: a) Adjust for reused objects

12. According to the Putnam model, approximately 40% of total project effort goes to development and 60% goes to maintenance.

- a) True b) False

Answer: True – This is the Putnam model rule of thumb.

13. Tom Cargill's famous quote states:

- a) "Good judgment comes from experience"
- b) "The first 90% of the code accounts for the first 90% of the time. The remaining 10% accounts for the other 90%."
- c) "Adding more people to a late project makes it later"
- d) "Software schedule slippage is easy to detect"

Answer: b) The 90/90 rule

14. Software schedule slippage, as it is happening, is easy to detect.

- a) True b) False

Answer: False – *It is HARD to detect as it is happening.*

Chapter 8: Project Planning & Controls

15. A Work Breakdown Structure (WBS) is a hierarchical decomposition of project work into manageable chunks.

- a) True b) False

Answer: True – Each level provides further definition and detail.

16. In a PERT chart, the critical path is identified by tasks that have:

- a) The longest duration
b) Zero slack (float) time
c) The most resources assigned
d) The highest cost

Answer: b) Slack = $L_i - F_i$. Zero slack = critical path.

17. If an activity on the critical path is delayed, the entire project will be delayed.

- a) True b) False

Answer: True – Defining characteristic of the critical path.

18. Schedule Variance (SV) is calculated as:

- a) $SV = AC - PV$
b) $SV = EV - AC$
c) $SV = EV - PV$
d) $SV = PV - EV$

Answer: c) $SV = BCWP - BCWS = EV - PV$

19. Cost Variance (CV) is calculated as:

- a) $CV = PV - AC$
b) $CV = EV - AC$
c) $CV = AC - EV$
d) $CV = EV - PV$

Answer: b) $CV = BCWP - ACWP = EV - AC$

20. A Cost Performance Index (CPI) of 0.89 means:

- a) The project is 89% complete
b) For every \$1 spent, you are getting 89 cents of planned value (over budget)
c) The project is 11% under budget
d) The project will finish 89% on time

Answer: b) $CPI < 1 = \text{over budget}$

21. A Schedule Performance Index (SPI) of 1.2 means:

- a) The project is 20% over budget
b) The project is 20% behind schedule
c) The project has 20% slack

d) The project is 20% ahead of schedule

Answer: d) $SPI > 1$ = ahead of schedule

22. Given $BAC = \$500K$ and $CPI = 0.80$, the Estimated Cost at Completion (EAC) is:

- a) \$400,000
- b) \$500,000
- c) \$600,000
- d) \$625,000

Answer: c) \$625,000 – $EAC = BAC/CPI = 500K/0.80$. *Note: answer is d if listed as \$625K.*

23. If a project has $BCWS = \$50K$, $BCWP = \$40K$, and $ACWP = \$45K$, the project is behind schedule AND over budget.

- a) True b) False

Answer: True – $SV = 40 - 50 = -10K$ (behind). $CV = 40 - 45 = -5K$ (over budget).

24. Risk Exposure is calculated as:

- a) Risk probability + Risk effect
- b) Risk probability $\times$ Risk effect
- c) Risk probability / Risk effect
- d) Risk effect – Risk probability

Answer: b) $RE = probability \times effect$

25. Most software projects do too much risk management.

- a) True b) False

Answer: False – *Most projects do too LITTLE risk management.*

26. Which of the following is the #1 risk factor in software projects according to Boehm's Top 10?

- a) Wrong user interface
- b) Unrealistic schedule
- c) Personnel shortfall
- d) Requirements volatility

Answer: c) Personnel shortfall

27. The Gantt chart was created by Henry Gantt in the 1910s and is used for visual timeline representation of project schedules.

- a) True b) False

Answer: True – *Mechanical engineer and management consultant.*

28. In Earned Value Management, what CANNOT be determined from just PV and AC alone?

- a) How much was spent
- b) How much was planned

- c) Whether the project is on schedule
- d) How much money remains

Answer: c) You need EV (BCWP) to determine schedule performance.

Chapter 13: Software Testing

29. A human mistake during coding is called a(n):

- a) Failure
- b) Fault
- c) Error
- d) Defect

Answer: c) Error – Error = human mistake. Fault = bug. Failure = bad behavior.

30. A bug/defect in the source code is called a(n):

- a) Error
- b) Failure
- c) Fault
- d) Crash

Answer: c) Fault – manifestation of an error in software.

31. Testing can show the presence of errors but NOT their absence.

- a) True
- b) False

Answer: True – Fundamental principle of testing.

32. “Verification” in software testing means:

- a) Building the right product
- b) Building the product right
- c) Testing by the customer
- d) Running all test cases

Answer: b) Verification = product right (against specs).

33. “Validation” in software testing means:

- a) Building the product right
- b) Building the right product
- c) Running unit tests
- d) Code inspection

Answer: b) Validation = right product (customer needs).

34. Which testing level tests individual program units in isolation?

- a) System testing
- b) Integration testing
- c) Acceptance testing
- d) Unit testing

Answer: d) Unit testing

35. Which coverage criterion is the WEAKEST?

- a) Branch/Decision coverage

- b) MC/DC
- c) Statement coverage (All-Nodes)
- d) All-Paths coverage

Answer: c) Statement coverage – Statement < Branch < MC/DC < All-Paths

36. Modified Condition/Decision Coverage (MC/DC) is required for:

- a) Web applications
- b) Mobile apps
- c) Safety-critical aviation software (DO-178)
- d) Database systems

Answer: c) DO-178 standard for safety-critical software.

37. In a Fagan Inspection, the code author actively leads the review and defends the code.

- a) True
- b) False

Answer: False – *The author is a SILENT observer. The moderator leads.*

38. In Test-Driven Development (TDD), tests are written AFTER the code is implemented.

- a) True
- b) False

Answer: False – *TDD: write tests FIRST, then create code to pass them.*

39. In mutation testing, the adequacy score is calculated as:

- a) Total mutants / Dead mutants
- b) Live mutants / Total mutants
- c) Dead mutants / Total mutants
- d) Total tests / Dead mutants

Answer: c) Higher score = better test set.

40. Cost of error correction increases EXPONENTIALLY from requirements through operation phases.

- a) True
- b) False

Answer: True – *Finding bugs later costs exponentially more.*

41. Regression testing is performed to:

- a) Test new features only
- b) Ensure changes do not break existing functionality
- c) Replace unit testing
- d) Satisfy customer acceptance criteria

Answer: b) Retest after changes to prevent regressions.

42. No single test technique is universally best; using multiple techniques discovers more faults.

- a) True
- b) False

Answer: True – *Fagan inspections are very cost-effective.*

Chapter 14: Software Maintenance

43. Which type of maintenance accounts for the LARGEST portion of maintenance effort?

- a) Corrective (fixing bugs)
- b) Adaptive (environment changes)
- c) Perfective (new/changed user requirements)
- d) Preventive (improve maintainability)

Answer: c) Perfective = 50%, Adaptive=25%, Corrective=21%, Preventive=4%.

44. Preventive maintenance is sometimes called “Refactoring” in Agile/XP development.

- a) True b) False

Answer: True – Same concept: improve maintainability without changing functionality.

45. In the effort distribution of maintenance tasks, which activity takes the MOST effort?

- a) Modification (30%)
- b) Testing (30%)
- c) Isolation / determining what to change (40%)
- d) Documentation (10%)

Answer: c) Isolation = 40%

46. Lehman’s Law of Increasing Complexity states:

- a) Programs become simpler as they evolve
- b) A changed program becomes less structured and more complex over time
- c) Programs maintain their structure through maintenance
- d) Complexity decreases with preventive maintenance

Answer: b) Extra effort required to avoid increasing complexity.

47. Reverse engineering involves adapting and changing the system to add new functionality.

- a) True b) False

Answer: False – RE = reconstruct design from existing system. Does NOT change it.

48. What is the difference between Reengineering and Restructuring?

- a) Reengineering changes functionality; Restructuring does not
- b) Restructuring changes functionality; Reengineering does not
- c) They are the same thing
- d) Neither changes functionality

Answer: a) Reengineering = RE + FE, functionality changes. Restructuring = no change.

49. Software maintenance requires LESS skill than development because the code already exists.

- a) True b) False

Answer: False – Maintenance requires MORE skill: must understand others’ code AND write new code.

50. A “beacon” in program comprehension is a key feature that helps a maintainer form hypotheses about what the code does (e.g., a swap operation suggests sorting).

- a) True b) False

Answer: True – *Beacons are key features recognized by experienced programmers.*

ANSWER KEY – QUICK REFERENCE

- | | |
|-----------|-----------|
| 1. True | 26. c |
| 2. False | 27. True |
| 3. a | 28. c |
| 4. c | 29. c |
| 5. False | 30. c |
| 6. b | 31. True |
| 7. a | 32. b |
| 8. a | 33. b |
| 9. False | 34. d |
| 10. c | 35. c |
| 11. a | 36. c |
| 12. True | 37. False |
| 13. b | 38. False |
| 14. False | 39. c |
| 15. True | 40. True |
| 16. b | 41. b |
| 17. True | 42. True |
| 18. c | 43. c |
| 19. b | 44. True |
| 20. b | 45. c |
| 21. d | 46. b |
| 22. c | 47. False |
| 23. True | 48. a |
| 24. b | 49. False |
| 25. False | 50. True |